Project: Milestone 1 Deliverables and Checklist

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

How to submit

All M1 deliverables are submitted by pushing to your public GitHub repository. There is no Canvas upload. The grader will scrape your repository at 11:59 pm on the due date and check for each file at the exact path listed in this checklist. A missing file at the expected path counts as Not Yet, even if the content exists elsewhere in the repository.

Expected repository structure for M1

Your repository should contain at minimum the following layout after M1. Paths are relative to the repository root.

```
your-repo/
├── README.md
├── project/
│   ├── heilmeier.md                      ← Heilmeier Q1-Q3
│   ├── algorithm_diagram.png             ← from Codefest 1
│   └── m1/
│       ├── sw_baseline.md                ← software benchmark
│       ├── interface_selection.md        ← interface choice + bandwidth analysis
│       └── system_diagram.png            ← chiplet block diagram
└── codefest/
    └── cf02/
        ├── profiling/
        │   ├── project_profile.txt       ← profiler output
        │   └── roofline_project.png      ← roofline plot
        └── analysis/
            ├── ai_calculation.md         ← arithmetic intensity
            └── partition_rationale.md    ← HW/SW partition
```

The `project/m1/` folder is new. Create it now if it does not exist. The `codefest/cf02/` paths were established during Codefest 2.

Deliverable checklist

Check off each item only when the file is committed and pushed. A checked box that points to a missing or empty file is still Not Yet.

Deliverable	GitHub file path
1. Heilmeier questions (Q1, Q2, Q3)	
☐ Refined answer to Q1: What are you trying to do? *All three questions in one file. Be specific: name the algorithm and the kernel.*	`project/heilmeier.md`
☐ Refined answer to Q2: What is done today and what are the limits? *Must reference your profiling data. A vague answer here signals the scoping is not ready.*	`project/heilmeier.md`
☐ Refined answer to Q3: What is your approach and why is it better? *Ground this in your arithmetic intensity result and roofline position.*	`project/heilmeier.md`
2. Software baseline benchmark	
☐ Platform and configuration documented (CPU/GPU model, OS, Python version, batch size) *The M4 speedup comparison must be reproducible from this information.*	`project/m1/sw_baseline.md`
☐ Execution time measured (wall-clock, median over ≥10 runs)	`project/m1/sw_baseline.md`
☐ Throughput reported (samples/sec, tokens/sec, or FLOPs/sec as appropriate)	`project/m1/sw_baseline.md`
☐ Memory usage reported (peak RSS or GPU memory as appropriate)	`project/m1/sw_baseline.md`
3. Profiling and dominant kernel identification	
☐ Profiler output saved (cProfile, line_profiler, or torch.profiler) *Raw output is fine. Must show per-function or per-layer timing.*	`codefest/cf02/profiling/project_profile.txt`
☐ Dominant kernel named explicitly with percentage of total runtime stated *One sentence minimum: "The dominant kernel is X, accounting for Y% of total runtime."*	`codefest/cf02/analysis/ai_calculation.md`
4. Arithmetic intensity calculation	
☐ FLOPs counted analytically with formula and substituted values shown	`codefest/cf02/analysis/ai_calculation.md`

Derive from the algorithm, not from a profiler estimate. Show the formula first, then the numbers.	
☐ Bytes transferred estimated assuming DRAM access with no reuse, shown explicitly *Separate the operand types: weights, inputs, outputs.*	`codefest/cf02/analysis/ai_calculation.md`
☐ Arithmetic intensity value stated in FLOP/byte	`codefest/cf02/analysis/ai_calculation.md`
5. Roofline plot	
☐ Target hardware platform identified with peak FLOP/s and peak memory bandwidth cited from spec sheet *State the source (e.g., Intel ARK, NVIDIA spec sheet). Do not guess.*	`codefest/cf02/profiling/roofline_project.png`
☐ Ridge point computed and labeled on plot *Ridge point = peak FLOP/s ÷ peak bandwidth (in FLOP/byte).*	`codefest/cf02/profiling/roofline_project.png`
☐ Software kernel plotted and labeled at its computed arithmetic intensity	`codefest/cf02/profiling/roofline_project.png`
☐ Hypothetical HW accelerator design point plotted and labeled *Choose a target compute throughput and on-chip bandwidth. Justify the choice in the partition rationale.*	`codefest/cf02/profiling/roofline_project.png`
☐ Axes labeled (x: FLOP/byte, y: GFLOP/s), both log scale	`codefest/cf02/profiling/roofline_project.png`
6. HW/SW partition proposal	
☐ Which kernel(s) to accelerate in hardware, with roofline justification	`codefest/cf02/analysis/partition_rationale.md`
☐ What remains in software and why	`codefest/cf02/analysis/partition_rationale.md`
☐ Whether the kernel is currently compute-bound or memory-bound, and whether the HW design changes that	`codefest/cf02/analysis/partition_rationale.md`
☐ Interface bandwidth required to avoid the accelerator becoming interface-bound (with calculation) *Minimum: state the required bandwidth in GB/s and compare it to your chosen interface's rated bandwidth.*	`codefest/cf02/analysis/partition_rationale.md`
☐ Document is ≥200 words	`codefest/cf02/analysis/partition_rationale.md`
7. Interface selection	

☐ One interface chosen from the project document table (SPI, I²C, AXI4-Lite, AXI4 Stream, PCIe, UCIe)	`project/m1/interface_selection.md`
☐ Bandwidth requirement of the kernel at the target operating point calculated *Show the calculation: throughput × data width = required bandwidth in GB/s.*	`project/m1/interface_selection.md`
☐ Interface rated bandwidth compared to required bandwidth; bottleneck status stated *Is your design interface-bound on the roofline? If so, quantify the impact.*	`project/m1/interface_selection.md`
☐ Host platform assumed stated explicitly (MCU, FPGA SoC, data-center host, etc.)	`project/m1/interface_selection.md`
8. High-level system block diagram	
☐ Diagram shows: host, interface, chiplet boundary, compute engine, on-chip memory *Accepted formats: .png, .pdf, .jpg. Hand-drawn and photographed is fine.*	`project/m1/system_diagram.png`
☐ All five components labeled clearly *The chiplet boundary must be visually distinct (e.g., a box or dashed line).*	`project/m1/system_diagram.png`

Common reasons for Not Yet

The following are the most frequent reasons an M1 submission receives Not Yet. Review each before submitting.

Missing or wrong file path

The grader checks exact paths. A file at `project/M1/sw_baseline.md` will not be found if the expected path is `project/m1/sw_baseline.md`. Use lowercase, match the paths in this checklist exactly.

Heilmeier Q2 not grounded in data

"Current software is slow" is not an acceptable answer to Q2. The answer must reference a specific runtime measurement or bottleneck identified in profiling. If you cannot answer Q2 with data, your M1 is not ready.

Arithmetic intensity shown as a single number without derivation

Stating "AI = 3.2 FLOP/byte" without showing the FLOPs formula and byte count is Not Yet. The calculation must be reproducible from what you write.

Roofline plot missing the hypothetical HW design point

The roofline must show two points: where your software kernel sits today, and where you intend your accelerator to sit. A plot with only the software kernel does not satisfy the M1 requirement.

Interface selection without a bandwidth calculation

"I chose AXI4 Stream because it is fast" is not a justification. You must compute the bandwidth your kernel requires and compare it to the interface's rated bandwidth.

SW baseline not reproducible

If the platform (hardware model, OS, Python version, batch size) is not documented, the M4 comparison cannot be verified. Document everything needed for someone else to reproduce the number.